

Hybrid Threat Detection at the Application Layer

A dual-stream neural network for detecting phishing and SQL injection text

Luis Moto · Neural networks course project / portfolio piece · Built with Keras & TensorFlow

What this is

This is the final project I built for my neural networks course, and I'm keeping it in my portfolio because the architecture taught me a lot about combining different types of input in one model. It's a classifier that looks at a piece of text — an email or a web request — and decides whether it's safe, a phishing attempt, or a SQL injection attack.

It is **not** a production security system. It was trained and evaluated on public datasets (combined from Kaggle), not live traffic, and it hasn't been tested against adversarial or real-world adaptive attacks. I'm presenting it here as a learning project and a demonstration of architecture design, not as something ready to sit in front of a real application.

The problem I wanted to explore

Phishing and SQL injection are usually detected one of two ways: rigid regex/signature rules, which break the moment an attacker changes the payload slightly, or large language models, which understand text well but are often too heavy and too "smoothed out" for this kind of task — many of them treat characters like `'`, `--`, or `=` as noise, when in security text those exact characters are often the attack itself.

I wanted to see if I could get the best of both approaches: a model that understands text semantically, but that also looks directly at structural, rule-based signals — without throwing away the special characters that actually matter. That's what led me to a dual-stream design instead of a single model.

Architecture

The model has two parallel branches that process the same input differently, then merge before the final classification layer. I built it with the Keras Functional API, since a simple sequential model can't express two separate inputs joining mid-network.

1. Text branch (semantic stream)

- **Custom embedding, trained from scratch.** I deliberately avoided a pretrained model like BERT for this. Most pretrained tokenizers strip out exactly the punctuation that matters for this task (`'`, `--`, `=`, `*`). I used a closed vocabulary of 8,000 tokens and a fixed sequence length of 256.
- **Multi-head self-attention** (2 heads, key dimension 32) to let the model weigh relationships between tokens, not just read them in order.
- **Global max pooling** at the end of this branch, so a short malicious payload (like `admin' OR 1=1 --`) doesn't get diluted if it's surrounded by a lot of ordinary text.

2. Structural branch (engineered features)

In parallel, a second branch takes a 14-dimensional vector of handcrafted features — things like keyword density, URL structure anomalies, and other regex-derived signals. I added batch normalization here because these features mix continuous values (0–1 ranges) with binary flags, and the scale difference was hurting training stability before I added it.

The two branches are concatenated, passed through a dense layer with dropout, and end in a 3-class softmax output: safe, phishing, or SQL injection.

Model definition

```
# Semantic Branch: Text Sequence Processing
text_input = Input(shape=(256,), name="text_input", dtype=tf.int32)
embedding = layers.Embedding(input_dim=8000, output_dim=64)(text_input)

# Transformer block with residual connection and layer normalization
attn_output = layers.MultiHeadAttention(num_heads=2, key_dim=32)(embedding, embedding)
attn_output = layers.Dropout(0.1)(attn_output)
out1 = layers.Add()([embedding, attn_output])
out1 = layers.LayerNormalization(epsilon=1e-6)(out1)

# Sequence reduction and semantic compression
text_vec = layers.GlobalMaxPooling1D()(out1)
text_dense = layers.Dense(16, activation="relu")(text_vec)

# Structural Branch: Feature Vector Processing
feat_input = Input(shape=(14,), name="feat_input", dtype=tf.float32)
feat_dense = layers.Dense(32, activation="relu")(feat_input)
feat_dense = layers.BatchNormalization()(feat_dense)
feat_out = layers.Dense(16, activation="relu")(feat_dense)

# Fusion and classification
concat = layers.Concatenate()([text_dense, feat_out])
concat = layers.Dropout(0.2)(concat)
```

```
concat = layers.Dense(64, activation="relu")(concat)
outputs = layers.Dense(3, activation="softmax", name="output")(concat)

model = tf.keras.Model(inputs=[text_input, feat_input], outputs=outputs)
```

Inference function

```
MODEL = tf.keras.models.load_model('neural_threat_analyzer.keras')
with open('vectorizer_config.pkl', 'rb') as f:
    VEC_DATA = pickle.load(f)

VECTORIZER = tf.keras.layers.TextVectorization.from_config(VEC_DATA['config'])
VECTORIZER.set_vocabulary(VEC_DATA['vocabulary'])

def predict_threat(raw_text: str) -> dict:
    if not raw_text.strip():
        return {"prediction": -1, "threat_name": "Empty input", "confidence": 0.0}

    clean_text = security_preprocess(raw_text)
    vec_text = VECTORIZER([clean_text])
    manual_feats = np.array([extract_features(clean_text)], dtype=np.float32)

    y_pred_probs = MODEL.predict([vec_text, manual_feats], verbose=0)
    prediction = int(np.argmax(y_pred_probs, axis=1)[0])
    confidence = float(np.max(y_pred_probs) * 100)

    threat_classes = {0: "Safe Content", 1: "Phishing Attack", 2: "SQL Injection"}
    return {
        "prediction": prediction,
        "threat_name": threat_classes.get(prediction),
        "confidence": confidence,
        "probabilities": {threat_classes[i]: float(p * 100) for i, p in enumerate(y_pred_probs[0])}
    }
```

Data

The dataset was built by combining public sources from Kaggle into a single corpus of 34,020 samples covering the three classes. The original class distribution was imbalanced toward "safe" text, so I downsampled to a symmetric 11,340 samples per class to avoid the model defaulting to a "probably safe" guessing strategy.

Split	Proportion	Samples	Purpose
Train	60%	20,412	Backpropagation / weight tuning
Validation	20%	6,803	Overfitting monitoring, EarlyStopping
Test	20%	6,805	Final, held-out evaluation

Worth being upfront about: this is curated, public benchmark data — not live production traffic. Real-world inputs are messier, more adversarial, and shift over time in ways a static Kaggle dataset doesn't capture. The numbers below describe performance on this test set specifically, not a guarantee of real-world performance.

Results

Accuracy (held-out test set)	99.13%
Macro F1-score	99.13%
Cross-validation mean accuracy	98.98%
Mean inference latency	< 15 ms

Per-class metrics (test set, 6,805 samples)

Class	Precision	Recall	F1-score	Support
Safe (0)	98.09%	99.43%	98.75%	2,268
Phishing (1)	99.47%	98.90%	99.18%	2,268
SQL Injection (2)	99.87%	99.07%	99.47%	2,269
Macro avg	99.14%	99.13%	99.13%	6,805

Confusion matrix (5,103 samples — predictions.csv)

Rows = true class · Columns = predicted class. Off-diagonal cells are misclassifications.

	Predicted: Safe (0)	Predicted: Phishing (1)	Predicted: SQL Inj. (2)
Actual: Safe (0)	1,689	11	1
Actual: Phishing (1)	24	1,673	4
Actual: SQL Inj. (2)	2	1	1,698

Total misclassifications: 43 out of 5,103. The most common error is safe content predicted as phishing (24 cases) — a false positive that errs on the side of caution. SQL injection is the most precisely separated class, with only 3 misclassifications total.

What I'd want to test next

- Adversarial and obfuscated inputs that go beyond what's represented in the public datasets used here.
- Performance drift on real, messier traffic instead of curated benchmark text.
- Latency and memory behavior under realistic concurrent load, rather than a single-instance test.
- Whether the 14 engineered features generalize to attack patterns not present in the training data.

What I took away from this

The main lesson wasn't the accuracy number — it was learning to design a model topology around what the problem actually needs, rather than reaching for a generic pretrained model by default. Building the custom embedding instead of using BERT, specifically to preserve characters that pretrained tokenizers discard, was the most useful architectural decision in the project, and the one I'm most glad I made the effort to understand rather than skip.